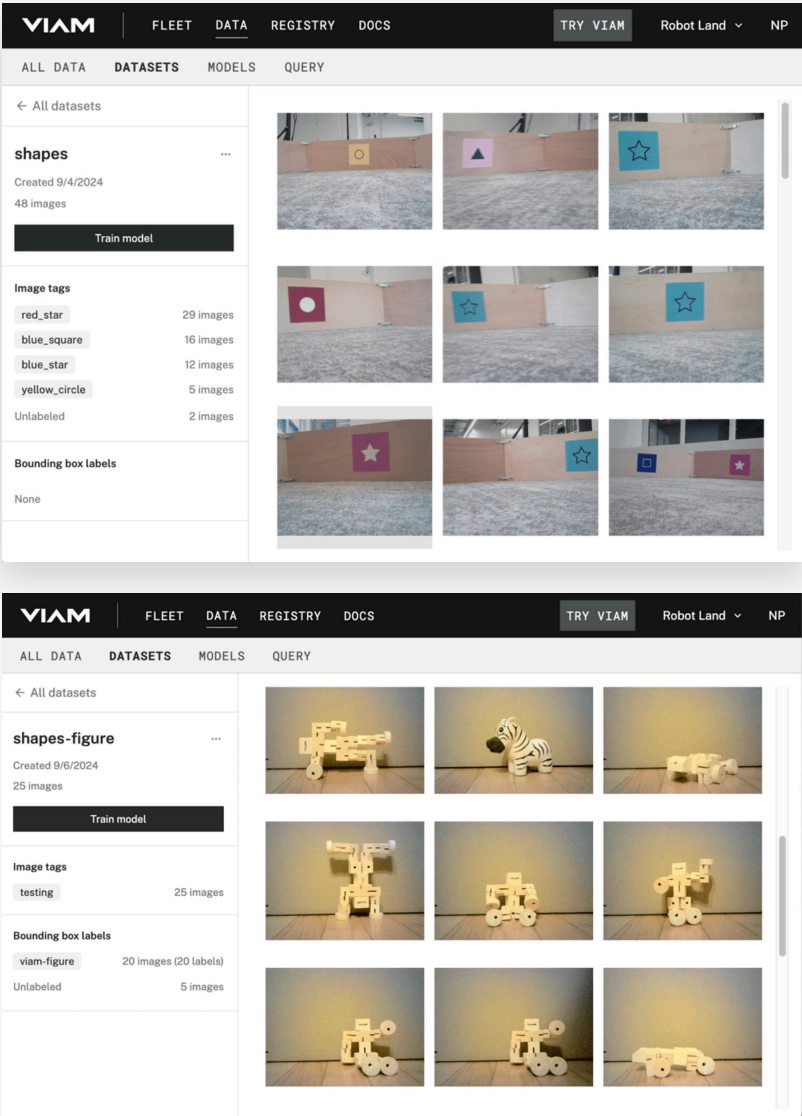


To ensure a machine learning model you create performs well, you need to train it on a variety of images that cover the range of things your machine should be able to recognize.

This page will walk you through labeling images for machine learning and creating a dataset with them.

Just testing and want a dataset to get started with? Click here.

We have two datasets you can use for testing, one with shapes and the other with a wooden figure:



1. [Download the shapes dataset](#) or [download the wooden figure dataset](#).
2. Unzip the download.
3. Open a terminal and go to the dataset folder.
4. Create a python script in the dataset’s folder with the following contents:

```

1  # Assumption: The dataset was exported using the `viam dataset export` command.
2  # This script is being run from the `destination` directory.
3
4  import asyncio
5  import os
6  import json
7  import argparse
8
9  from viam.rpc.dial import DialOptions, Credentials
10 from viam.app.viam_client import ViamClient
11 from viam.proto.app.data import BinaryID
12
13 async def connect(args) -> ViamClient:
14     dial_options = DialOptions(
15         credentials=Credentials(
16             type="api-key",
17             payload=args.api_key,
18         ),
19         auth_entity=args.api_key_id
20     )
21     return await ViamClient.create_from_dial_options(dial_options)
22
23
24 async def main():
25     parser = argparse.ArgumentParser(
26         description='Upload images, metadata, and tags to a new dataset')
27     parser.add_argument('-org-id', dest='org_id', action='store',
28                         required=True, help='Org Id')
29     parser.add_argument('-api-key', dest='api_key', action='store',
30                         required=True, help='API KEY with org admin access')
31     parser.add_argument('-api-key-id', dest='api_key_id', action='store',
32                         required=True, help='API KEY ID with org admin access')
33     parser.add_argument('-machine-part-id', dest='machine_part_id',
34                         action='store', required=True,
35                         help='Machine part id for image metadata')
36     parser.add_argument('-location-id', dest='location_id', action='store',
37                         required=True, help='Location id for image metadata')
38     parser.add_argument('-dataset-name', dest='dataset_name', action='store',
39                         required=True,
40                         help='Name of the data to create and upload to')
41     args = parser.parse_args()
42
43
44     # Make a ViamClient
45     viam_client = await connect(args)
46     # Instantiate a DataClient to run data client API methods on
47     data_client = viam_client.data_client
48
49     # Create dataset
50     try:
51         dataset_id = await data_client.create_dataset(
52             name=args.dataset_name,
53             organization_id=args.org_id
54         )
55         print("Created dataset: " + dataset_id)
56     except Exception:
57         print("Error. Check that the dataset name does not already exist.")
58         print("See: https://app.viam.com/data/datasets")
59         return 1
60
61     file_ids = []
62
63     for file_name in os.listdir("metadata/"):
64         with open("metadata/" + file_name) as f:
65             data = json.load(f)
66             tags = None
67             if "tags" in data["captureMetadata"].keys():
68                 tags = data["captureMetadata"]["tags"]
69
70             annotations = None
71             if "annotations" in data.keys():
72

```

```

72         annotations = data["annotations"]
73
74     image_file = data["fileName"]
75
76     print("Uploading: " + image_file)
77
78     id = await data_client.file_upload_from_path(
79         part_id=args.machine_part_id,
80         tags=tags,
81         filepath=os.path.join("data/", image_file)
82     )
83     print("FileID: " + id)
84
85     binary_id = BinaryID(
86         file_id=id,
87         organization_id=args.org_id,
88         location_id=args.location_id
89     )
90
91     if annotations:
92         bboxes = annotations["bboxes"]
93         for box in bboxes:
94             await data_client.add_bounding_box_to_image_by_id(
95                 binary_id=binary_id,
96                 label=box["label"],
97                 x_min_normalized=box["xMinNormalized"],
98                 y_min_normalized=box["yMinNormalized"],
99                 x_max_normalized=box["xMaxNormalized"],
100                 y_max_normalized=box["yMaxNormalized"]
101             )
102
103         file_ids.append(binary_id)
104
105     await data_client.add_binary_data_to_dataset_by_ids(
106         binary_ids=file_ids,
107         dataset_id=dataset_id
108     )
109     print("Added files to dataset.")
110     print("https://app.viam.com/data/datasets?id=" + dataset_id)
111
112     viam_client.close()
113
114 if __name__ == '__main__':
115     asyncio.run(main())

```

5. Run the script to upload the images and their metadata into a dataset in Viam app providing the following input:

```

$ python upload_data.py -org-id <ORG-ID> -api-key <API-KEY> \
$ -api-key-id <API-KEY-ID> -machine-part-id <MACHINE-PART-ID> \
$ -location-id <LOCATION-ID> -dataset-name <NAME>

```

6. Continue to [Train a tflite machine learning model](#).

Prerequisites

✓ At least 10 captured images of what you want your machine to recognize.

[Capture and sync image data](#) using the data management service.

When training machine learning models, it is important to supply a variety of images. The dataset you create should represent the possible range of visual input. This may include capturing images of different angles, different configurations of objects and different lighting conditions. The more varied the provided dataset, the more accurate the resulting model becomes.

Tips on improving model accuracy

- **More data means better models:** Incorporate as much data as you practically can to improve your model’s overall performance.
- **Include counterexamples:** Include images with and without the object you’re looking to classify. This helps the model distinguish the target object from the background and reduces the chances of false positives by teaching it what the object is not.
- **Avoid class imbalance:** Don’t train excessively on one specific type or class, make sure each category has a roughly equal number of images. For instance, if you’re training a dog detector, include images of various dog breeds to avoid bias towards one breed. An imbalanced dataset can lead the model to favor one class over others, reducing its overall accuracy.
- **Match your training images to your intended use case:** Use images that reflect the quality and conditions of your production environment. For example, if you plan to use a low-quality camera in production, train with low-quality images. Similarly, if your model will run all day, capture images in both daylight and nighttime conditions.
- **Vary your angles and distances:** Include image examples from every angle and distance that the model will see in normal use.
- **Ensure labeling accuracy:** Make sure the labels or bounding box annotations you give are accurate.

Label your images

Once you have enough images, you can disable data capture to [avoid incurring fees](#) for capturing large amounts of training data.

Then use the interface on the [DATA tab](#) to label your images.

Most use cases fall into one of two categories:

- Detecting certain objects and their location within an image. For example, you may wish to know where and how many **pizzas** there are in an image. In this case, use the **Annotate** interface to add a label for each object you would like to detect.

▼ For instructions to add labels, click here.

To add a label, click on an image and select the **Annotate** option in the menu that opens. Choose an existing label or create a new label. Holding the command key, click on the image where you would like to add the bounding box and drag to where the bounding box should end.

VIAM

FLEETDATAREGISTRYDOCS

TRY VIAMSierra's OrgSG

ALL DATA

DATASETS

TRAINING

MODELS

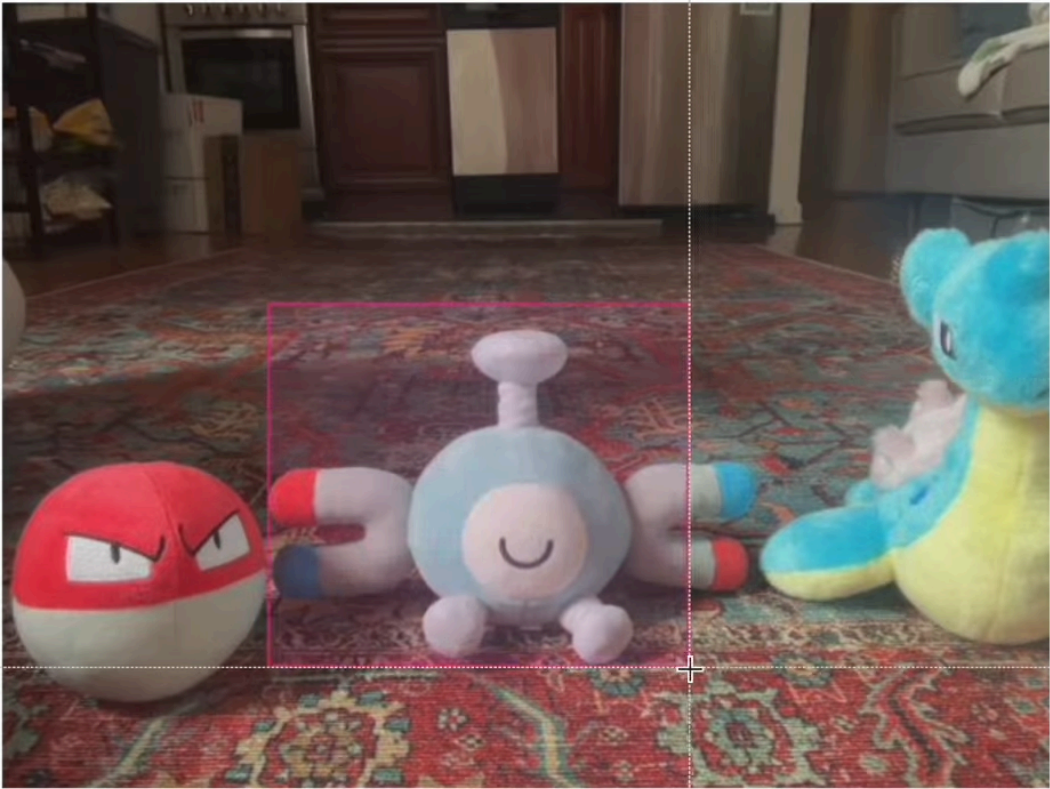
QUERY

✕

Labelmagnemite

Hold ⌘ to draw a bounding box

←



→



OBJECTS

Choose a label and draw a bounding box

TAGS

None+

Recent: full

DATASETS

None+

PREDICT CLASSIFICATIONS

Model name

Select model

Minimum confidence threshold ⓘ

0.0

?

You can move around or resize the bounding box once you have created it.

Repeat this with all images.

You can add one or more bounding boxes for objects in each image.

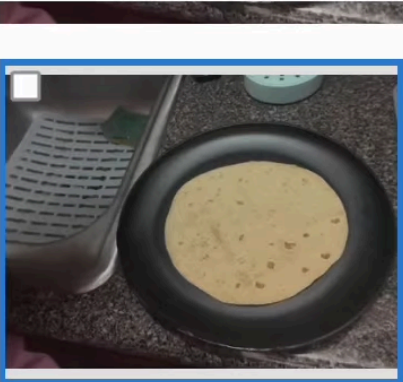
- Classifying an image as a whole. In other words, determining a descriptive state about an image. For example, you may wish to know whether an image of a food display is `full` , `empty` , or `average` or whether the quality of manufacturing output is `good` or `bad` . In this case, add tags to describe your images.

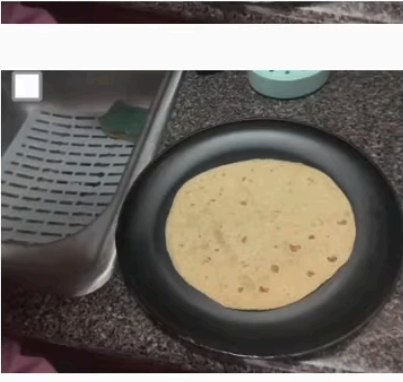
For instructions to add tags, click here.

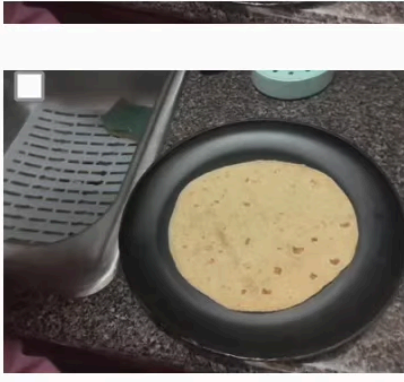
To tag an image, click on an image and click the `+` next to the **Tags** option. Add one or more tags to your image.

ImagesFilesPoint cloudsSensors

Delete selected









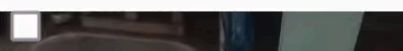


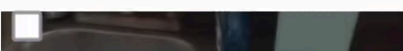














←×



ACTIONSDETAILS

Annotate

Objects

None

Tags

None

Choose or create...

tag_1tag_2TAG_3

Repeat this with all images.

Organize data into a dataset

To train a model, your images must be in a dataset.

Use the interface on the **DATA** tab to add your labeled images to a dataset.

Also add any unlabelled images to your dataset. Unlabelled images must not comprise more than 20% of your dataset. If you have 25 images in your dataset, at least 20 of those must be labelled.

Want to add images to a dataset programmatically? Click here.

You can also add all images with a certain label to a dataset using the `viam dataset data add` [command](#) or the [Data Client API](#):

CLIData Client API

```
$ viam dataset create --org-id=<org-id> --name=<name>
$ viam dataset data add filter --dataset-id=<dataset-id> --tags=red_star,blue_square
```

NEXT

Train TFlite model

